

A Modular and Continuously Improving Chatbot for Movie Recommendation

Trung Hien Nguyen¹, Trung Duc Do¹, Minh Hieu Dang Vu¹,
Tuan Nghia Le¹

University of Engineering and Technology - Vietnam National
University, Hanoi, Vietnam.

Contributing authors: 23020063@vnu.edu.vn; 23020045@vnu.edu.vn;
23020066@vnu.edu.vn; 23020124@vnu.edu.vn;

Abstract

This paper proposes a modular and continuously improving chatbot for movie recommendation. The overall system is the combined of routing technique in modular architecture and a framework to improve chatbot for movie recommendation time by time. The heart of the proposed architecture is the Node.js server, which decide whether to call TMDB API or Python server for machine learning algorithms. With the support of LLM, the suggested framework introduced 4 main stage from analyze intent characteristic, generate test cases, evaluate test suite to verify and enrich training phrases. The result show that the combined approach improve the intent classification accuracy from 72% to approximately 86%, fast response time, and improved scalability.

Keywords: chatbot, recommendation, Dialogflow, LLMs, orchestration

1 Introduction

Conversational recommender systems have become a popular research topic due to the natural and effective support they provide for users to discover items, compared to traditional filter-based approaches [3]. In the movie recommendation domain, chatbots built on frameworks such as **Dialogflow** are known for their simplicity and ease of deployment. However, many of these systems usually lack rich visuals, and training data remain unchanged over time, which limits their ability to handle new request or diverse expressions from users [1, 4].

To address these limitations, this paper proposes a modular and continuously improving chatbot for movie recommendations. The **Node.js** server is the heart of the system that automatically routes user requests to either **TMDB** or a **Python** backend for recommendation algorithms. Furthermore, we introduce a proposed framework that dynamically analyzes intent characteristics, generates test cases, evaluates test suites, and enriches training data. These improvements enable the chatbot to adapt over time while still maintaining the scalability and stability of the system in the real world environment.

2 Related Works

The implementation of conversational systems has been used in many fields to enhance user participation. These systems are often developed to overcome the rigidity of traditional keyword matching used in intent-based architectures. In this work, we build on these architectures, developing an adaptive movie recommendation agent that utilizes **Large Language Models** (LLMs) to handle conversational fallbacks and constant learning to improve recommendation quality.

Directly related to our system are systems designed specifically for movie recommendation relying on predefined intents, such as Vote Goat [1] and the Telegram-based chatbot [2]. While Vote Goat enables basic voice interaction and the Telegram-based chatbot employs maximum likelihood methods to handle local slang within the **Google Dialogflow** framework, both systems are constrained by the lack of deep contextual understanding inherent in intent-based models. In contrast to that, our system also provides a more dialogue mechanic through generative **LLM** fallbacks and a recommendation engine that can dynamically evolve based on users' responses.

3 Background

The movie recommendation is designed as a lightweight chatbot, which is being used to form a natural conversational search rather than rigid command interface. The system identifies user's intention through **Google's DialogFlow** framework to make user's input into structured actions that allow the system to work on. It judges the intention of the user's input by "intent", also DialogFlow do provides a way to customly train these intent based on specific context.[6].

Node.js server being used as a central orchestration layer, handling API, control routing and unifying data to respond.

Python Flask backend containing machine learning algorithms, which is a hybrid of content-based filtering and collaborative filtering. Content-based filtering is being used for recommends movies similar to those a user has liked in the past by analyzing the features of user's viewed, starred movies[7]. It involves Sentence Transformers to encode movie descriptions into parameters and FAISS to index them[8]. The collaborative filtering analyzes user behavior to recommend similar preferences[7], it uses Alternating Least Squares (ALS), Logistic Matrix Factorization (LMF), and Bayesian Personalized Ranking (BPR) with different weights to score items, then uses Thompson Sampling to sort out the actual output.

The movies' data is being fetched from **TMDB** (The Movie Database), a free, available database for movies and TV shows.

The system also has some uses on **Large language model (LLM)**, which in this case, use to test and confirm chatbot's output in natural language contexts.

4 The Proposed Method

4.1 Architecture

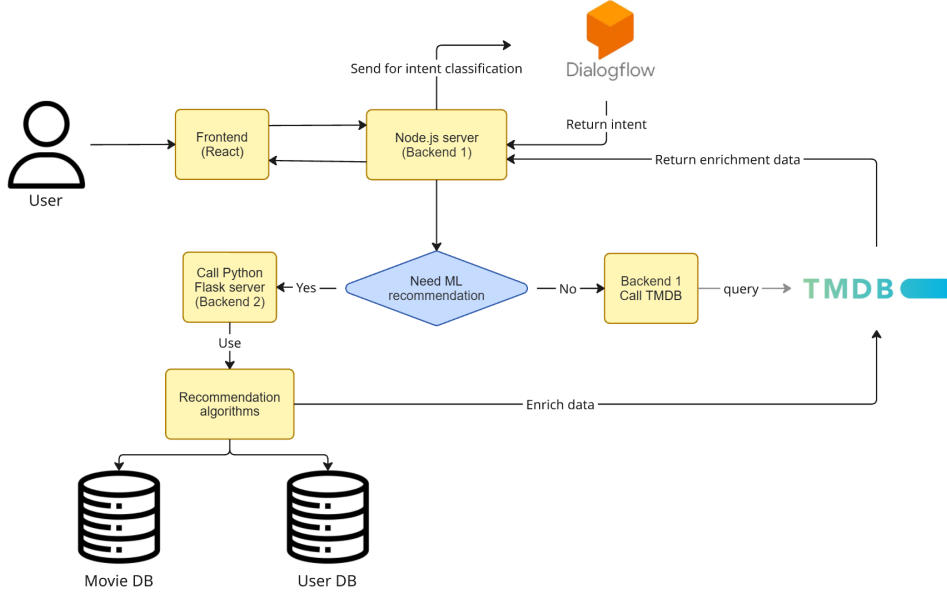


Fig. 1: Architecture

The end user interacts with the system via a web application built on **React**. The **Node.js** server (Backend 1) is the central orchestration layer. Backend 1 sends user messages received from the frontend to **Dialogflow** for intent classification and entity extraction. When the intent from **Dialogflow** arrives, Backend 1 will choose whether to call the **Python Flask** backend (Backend 2) for ML algorithms or to call **TMDB** for common recommendations. If intents that don't need to access user preferences, it will be handled directly by querying **TMDB**. This way ensure low latency and reduced overhead. Otherwise, intents that require user preferences or contextual understanding are sent to Backend 2 for further processing.

There are 2 types of recommendations in Backend 2: the first is a content-based recommendation for intent recommend_by_name, and the second is a collaborative filtering recommendation for intent recommend_personalization. In the proposed system, the content-based recommendation implement Sentence Transformers to encode movie

descriptions into dense semantic embeddings. After that, these embeddings are indexed using FAISS, enabling efficient semantic similarity search. For collaborative filtering, the system employs 3 models (ALS, BPR, LMF) to predict the recommendation score using an ensemble architecture. A film with a high recommendation score means user will prefer it. Each model contributes to the final recommendation score using different weights (ALS: 0.5, BPR: 0.3, LMF: 0.2). The last step is balance exploration and exploitation by implementing Thompson Sampling, this allows system to recommend a high range of film to users from high-confidence items to novel candidates.

A key innovation of the proposed system lies in its visual enrichment strategy. Both Backend 1 and Backend 2 are capable of querying **TMDB** to enrich movie metadata, including posters, ratings, and descriptions. Enrichment results will be sent back to Backend 1, which act as a unifying layer before forwarding final response to the frontend.

4.2 Framework

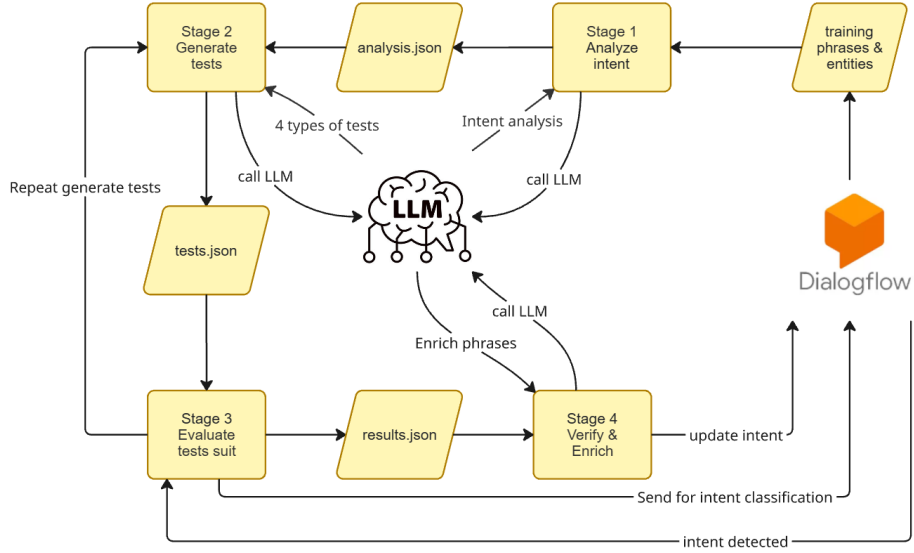


Fig. 2: Continuous Development Chatbot Framework

The main goal of the initial stage is to understand the characteristics and objectives of each intent and entity type from **Dialogflow**. After extracting the intent and entity list the system currently has, the proposed method applies intelligent sampling rather than analyzing all training phrases. This method uses only 10 training phrases in each intent to analyze, which helps avoid bias because some intents have many training phrases while others don't. For intents with required entities, we select 2 examples that are missing entities, 3 that are complete, and 5 random samples. For intents

without required entities, we implement stratified sampling to extract training phrases from low to high entity count. For entity sampling, we choose 10 examples based on synonym count and length. After a careful selection process, we leverage an **LLM** to analyze each intent by providing the intent name, parameters used, and training phrases as context. The output of the **LLM** is the analysis generated by **LLM**, it contains the intent’s goal, characteristics, entity_usage—the role of each entity in the intent, and key_patterns—the common word or structure of the intent.

In the following stage, the framework aims to create challenging test cases to expose classification weaknesses based on context from the previous stage. Using **analysis.json** from stage 1, we generate 12 test cases per intent across four categories: 5 paraphrasing, 3 ambiguous phrasings (using slang), 2 edge cases (phrases that has typos), and 2 phrases that have missing entities. All generated test cases are stored in **tests.json**. Every test case includes its corresponding metadata, such as test category, expected intent, and entities used in it.

In stage 3, each query in **tests.json** will be sent to **Dialogflow** to classify. It will be classified as a fail due to 3 main reasons. First, the predicted intent from **Dialogflow** differs from expected intent in **tests.json**. Second, the confidence intent predicted from **Dialogflow** is too low (below 0.5). The last reason is the entity caught by **Dialogflow** is different compared to the entity used from the phrase’s metadata. The outcome of each sentence will be written in **results.json**, which will be used to update the chatbot in stage 4. Another critical component of this framework is the iterative feedback loop, where failure cases will be sent back to **LLM** in stage 2. This allows **LLM** to generate larger, more comprehensive sentences to find the chatbot’s weaknesses.

The final stage focuses on verification and enrichment of the failure cases from **results.json**. The verify phase ensures the sentences that are failures in **results.json** are actually true, which can be done by either human or **LLM**. In the enrichment phase, each failure will be sent to **LLM** to create 5-10 variations applying the paraphrasing technique while still retaining semantics and required entities. After that, these phrases will be injected into the agent’s training set via API. Concluding with an automated retraining trigger.

5 Experiments

In order to validate the performance of the proposed approach, the experiment answers the following research questions:

- *RQ1*: How efficiently does the proposed architecture respond to user requests?
- *RQ2*: Does the continuous chatbot enhancement framework help the model learn more effectively from user requests?
- *RQ3*: Does the combination of architectural improvements and the proposed enhancement framework improve system stability and scalability?

These questions are investigated through experiments measuring response latency, intent classification errors across enrichment iterations, and system scalability.

5.1 Dataset

To create data for the chatbot, we use two different approaches for the training set and the testing set.

5.1.1 Training set

The dataset is mainly about conversations relating to movies. We have determined several intents that usually appear in users’ responses to the chatbot and accordingly extracted entities of these intents. Also, we created a training dataset manually to ensure high data quality and semantic consistency without noise or ambiguity. Each intent includes 22-25 training utterances.

5.1.2 Testing set

For the testing set, we implemented a different approach. A human-in-the-loop workflow combined with **LLMs** was utilized:

- **LLM-based generation:** We prompted LLMs to generate diverse utterances, including slang, complex sentence structures, and various phrasings that differ from the manually created training samples.
- **Human review:** Generated tests were reviewed by two people to remove hallucination and noise samples. The first person used LLMs to create a test suite from the previous stage, while the second person acted as a judge to determine if the test case was wrong or not. This approach helped the test suite not be biased and remain balanced.

This dataset is used to evaluate the system as a black box, assessing its ability to handle real inputs in different environments. This testing set is larger than the training set, consisting of approximately 300-500 utterances for each intent.

Dataset	Construction method	Scale (per intent)
Training set	Manual	22 - 25 utterances
Testing set	LLMs + Human verification	30 - 50 utterances

Table 1: Dataset distribution table

5.2 Model

The chatbot is developed based on **Dialogflow ES**, having built-in support for intents classification and entity recognition.

Baseline Model: A limitation of this preliminary approach is the lack of deep personalization in the recommendations, and also, the evaluations are still at an early stage of experimentation. It is also worth noting that the user’s chat experience is still heavily frame-based, with most of it relying on fixed intents and static sentences. As a result, chatbots are not able to adjust for people’s way of writing, which makes the model less flexible on handling unseen questions.

Proposed Model: To overcome the challenges of baseline approach, we propose a dynamic chatbot enhancement framework which learns from misclassified queries at each step. Rather than static training phrases, the chatbot gradually enriches its intents with false positives during LLM querying of user-like queries. It is easy from there to improve accuracy when having for instance to classify many different kinds of queries written in a lot of different ways.

5.3 Configuration

In this paper, we implement and deploy a chatbot using **Dialogflow** with its inbuilt machine learning for intent classification. The downside is that **Dialogflow** acts as a black-box; developers have little control on its internal hyperparameters. Therefore, our experimental setup is concentrated on the intent confidence threshold, which has a direct impact on what predicted intents are dealt with by the system.

The ML Classification Threshold decides if the predicted intent is reliable enough to be returned. When the confidence score is less than a given threshold, it launches the fallback intent. When there is no fallback intent, the algorithm selects nothing.

In our experiments, the threshold is 0.3. It is a trade-off between capturing as much user intentions and achieving more reliable intent classification so that the chatbot can deal with a variety of user requests while minimizing incorrect answers. To provide a fair comparison between the baseline model and the continuously enhanced one, we set the threshold value to stay constant in all experimental runs.

Parameter	Value
ML Classification Threshold	0.3

Table 2: Dialogflow configuration settings

5.4 Answer to research questions

5.4.1 Response Time Evaluation (RQ1)

In the proposed architecture, the Node.js server acts as an orchestration layer responsible for intent classification and routing requests. This approach helps reduce computational overhead from the Python-based recommendation backend. Concluding, boost efficiency, reduce response time, and increase the scalability of the system.

In table 3, while intent recommend_by_name has a high response time, 15 seconds if the model is cached and 120 seconds if the model is retrained, intent recommend_by_rating and intent recommend_by_genre only need 2 to 5 seconds to respond. The Node.js server ensures that only intents that require user preference or chat history are forwarded to the Python Flask backend. This significantly reduces unnecessary

Intent Name	Response Time (s)
Recommend by Genre	2-5
Recommend by Rating	2-5
Trending Movies	2-5
Movie Information	2-5
Recommend by Name (Content-based)	15-120
Personalized Recommendation (CF)	15-180

Table 3: Response time range of different intents

ML computation, leading to low latency and improved user experience. Node.js is also known for its ability to handle high concurrency; this helps the Python server to focus on high-computation activity such as collaborative filtering and semantic similarity search.

5.4.2 Continuously Enhanced Model (RQ2)

In the proposed method, user experiments will not be improved from their misclassified queries. Our method helps the chatbot better identify user intent, improving prediction accuracy as the system continues to learn.

In particular, we collect user queries and the LLM-generated answers that were misclassified and put them into a file called **failed_cases.json**. These mistaken queries are then manually edited to remove irrelevant or ambiguous ones that are not related to any present intent of the model. The rest of queries are paraphrased using multiple paraphrases that mean the same thing in order to augment the training data for each intent. Lastly, all finalized queries are included in the training phrases of their respective intents under **Dialogflow**, and one iteration of the chatbot enhancement process has been successfully completed.

To quantitatively evaluate the effectiveness of the proposed improvement framework, two metrics are used. The *Error Reduction Rate (%)* measures how many misclassified queries are reduced after each enhancement iteration, and is computed as the ratio between the number of reduced incorrect queries and the number of incorrect queries before improvement. The *Error Rate (%)* represents the proportion of misclassified queries over the total number of test queries, which is fixed at 350 utterances in our experiments.

Table 4 presents the experimental results obtained from the continuous improvement model on the testing set.

Iteration	Incorrect Queries (Before)	Incorrect Queries (After)	Error Reduction Rate (%)	Error Rate (%)	Enrich test cycle
1	97	86	11.34	24.57	1
2	86	67	21.83	16.62	1
3	67	58	13.43	16.62	3
4	58	50	13.79	14.33	1
5	50	51	-2.00	14.61	1
6	51	65	-27.45	18.62	5

Table 4: Experimental result table

As can be seen from the table, in each Step **1-4**, the model presents a step-wise decrease of the number of misclassified queries and lower error rate among all iterations. In the final two iterations, increasing the number of training sentences no longer leads to better performance. Instead, the growing amount of noisy data begins to outweigh the clean samples, resulting in a clear increase in both unclassified queries and overall error. These observations suggest that a model performs best when it is developed to an appropriate level of complexity; pushing it further can result in overfitting.

5.4.3 System Stability and Scalability (RQ3)

The proposed architecture improves the system’s scalability in multiple ways. The routing request technique in the **Node.js** backend enables parallel scaling. If users need more advanced recommendations, only the **Python** backend requires development, while if an orchestration layer becomes a bottleneck, only **Node.js** servers are scaled up. With the inclusion of the **TMDB** API, the costs to store movie metadata are decreased.

According to table 4, intent classification accuracy improved from 72% to 86%, which means users encounter fewer misclassification cases. As a result, boosting user experience and improving system reliability. Manual chatbot improvements require approximately 7 hours per iteration, while the proposed framework completes an iteration in approximately 15 minutes. Another improvement is that entities are detected in the new training phrase automatically, which helps reduce manual practice and improve the entity recognition capability of the original chatbot. These improvements allow the system to adapt to new intent, entity, or new domain without significant operational overhead. In conclusion, the proposed architecture and framework greatly boost system scalability and stability by reducing errors and time costs.

6 Conclusion

This paper introduced a modular chatbot system for movie recommendation based on **Dialogflow**, which is developed by combining a **Node.js** orchestration layer, a **Python-based** recommendation backend, and external services such as **TMDB**. Compared to a conventional intent-based chatbot with fixed training data, the proposed mining framework enables an incremental enhancement mechanism using LLMs for intent profiling, test case generation, and continuous expansion of training sentences. Experimental results indicate that this strategy provides more accurate intent classification, faster response times, and reduced human development effort. The use of a modular architecture makes it possible to scale the model since only complex queries are routed to the machine learning backend. Overall, the results show that adopting a modular system design and incorporating continuous learning is effective for building adaptive conversational recommendation systems.

References

- [1] J. Dalton, V. Ajayi, and R. Main, “Vote Goat: Conversational Movie Recommendation” in *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, 2018.
- [2] Z. A. Baizal, N. Ikhsan, I. M. K. Karo, R. K. Darmawan, and R. D. Hartanto, “Movie recommender chatbot based on Dialogflow” in *International Journal of Electrical and Computer Engineering (IJECE)*, 2023.
- [3] F. Ricci, L. Rokach, and B. Shapira, “Introduction to Recommender Systems Handbook,” in *Recommender Systems Handbook*, Springer, 2011.
- [4] M. McTear, “Conversational AI: Dialogue Systems, Conversational Agents, and Chatbots,” in *Synthesis Lectures on Human Language Technologies*, Morgan & Claypool Publishers, 2020.
- [5] Z. A. Baizal, N. Ikhsan, I. M. K. Karo, R. K. Darmawan, and R. D. Hartanto, “Movie Recommender Chatbot Based on Dialogflow,” in *International Journal of Electrical and Computer Engineering (IJECE)*, 2023.
- [6] Google Cloud, “Intents — Dialogflow ES,” Google Cloud Documentation, 2025. [Online]. Available: <https://docs.cloud.google.com/dialogflow/es/docs/intents-overview/>
- [7] GeeksforGeeks, “Content-based vs Collaborative Filtering: Difference,” GeeksforGeeks, 2025. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/content-based-vs-collaborative-filtering-difference/>
- [8] S. Ansari, “Unleashing the Power of Sentence Transformers: Revolutionising Semantic Search and Sentence Similarity,” Medium, Mar. 31, 2023.